FACYT - COMPUTACIÓN

20/NOV/2025

ORGANIZADOR DE GRILLA RADIAL

Estudiante:
JOSE TORRES

Descripción del proyecto

El proyecto consiste en desarrollar un sistema capaz de generar automáticamente la programación semanal de una emisora radial.

La problemática planteada establece que las emisoras organizan manualmente sus bloques de programación, lo que puede generar errores de distribución, repeticiones excesivas de canciones o mala ubicación de shows importantes.

El objetivo principal del software es automatizar la construcción de la grilla radial considerando:

- Canciones con distintos niveles de popularidad.
- Shows radiales con duración por segmentos.
- Publicidad obligatoria de patrocinantes.
- Horarios estelares.

Los horarios estelares definidos son:

- 07:00 – 09:00
- 12:00 – 14:00
- 18:00 – 19:00

En estos horarios se deben ubicar los shows de mayor rating para maximizar audiencia .

Además, el sistema permite consultas posteriores:

- Programación completa de un día.
- Elemento en reproducción en un momento exacto.
- Búsqueda de una canción específica.

---

3. Análisis de la solución

La solución fue diseñada bajo una arquitectura modular para facilitar mantenimiento, reutilización y legibilidad.

La estructura general fue separada en módulos:

radio.c

Archivo principal del programa.

Responsabilidades:

- Cargar archivos de entrada.
- Inicializar estructuras.
- Generar la programación semanal.
- Invocar menú principal.

---

radio.h

Archivo de cabecera.

Contiene:

- Definición de estructuras.
- Constantes globales.
- Prototipos de funciones.

Esto centraliza definiciones y evita duplicación.

---

menu_lib.c

Módulo de consultas.

Responsabilidades:

- Consulta por día.
- Consulta por momento específico.
- Consulta por canción.
- Menú principal.

Este módulo concentra toda la interacción con el usuario.

---

grilla_lib.c

Módulo de generación de programación.

Responsabilidades:

- Construcción automática de la grilla.
- Inserción de shows.

- Inserción de canciones.
- Inserción de publicidad.
- Control de repeticiones.

Justificación de diseño

Se modularizó porque:

- Reduce complejidad del `main()`.
- Aumenta legibilidad.
- Facilita depuración.
- Permite escalabilidad futura.

---

4. Estructuras de datos

Se utilizaron registros (`struct`) para modelar cada entidad.

Estructura canción

```
typedef struct{
    int seleccionado;
    char nombre[51];
    int min;
    int seg;
    int rate;
} cancion;
```

Función:

Representa una canción de la emisora.

Campos:

- nombre
- duración
- rating

Estructura patrocinador

```
typedef struct{
    int seleccionado;
    char nombre[31];
    int seg;
    int veces;
} patrocinador;
```

Función:

Representa una cuña publicitaria.

Campos:

- empresa
- duración
- repeticiones obligatorias

Estructura show

```
typedef struct{
    char nombre[101];
    int min;
    int seg;
    int segmentos;
    int rate;
    int seleccion;
} show;
```

Función:

Representa programas radiales.

Campos:

- nombre
- duración por segmento
- cantidad de segmentos
- rating

Estructura elemento

```
typedef struct{
    char nombre[101];
    int seleccionado;
    int min;
    int seg;
    char tipo;
} elemento;
```

Función:

Representa cualquier contenido en la grilla.

Tipos:

- C = canción
- S = show
- P = publicidad

Arreglo principal de programación

```
elemento grilla[SEGUNDOS_DIA];
```

Se usa para almacenar toda la programación del día.

Cada posición representa un segundo del día.

---

5. Algoritmos implementados

Lectura de archivos

Algoritmo:

1. Abrir archivo.
2. Leer línea por línea.
3. Separar nombre y valores numéricos.
4. Guardar en estructuras.

Pseudocódigo:

```
abrir archivo
mientras no EOF
   leer línea
   separar datos
   guardar datos
cerrar archivo
```

Ordenamiento por rating

Se implementó ordenamiento descendente.

Pseudocódigo:

```
para i = 0 hasta n-1
   para j = i+1 hasta n
      si rating[j] > rating[i]
         intercambiar
```

Se usa para priorizar mejores canciones y shows.

Generación de grilla

Proceso:

1. Insertar shows principales.
2. Completar espacios libres.
3. Elegir aleatoriamente:
    - canción
    - publicidad
    - show disponible

Pseudocódigo:

```
mientras tiempo < fin del día
   elegir tipo
   insertar elemento
   avanzar tiempo
```

Validación de repetición de canciones

Restricción:

Una canción no puede repetirse antes de 4 horas.

Pseudocódigo:

```
buscar 4 horas atrás y adelante
si existe
   rechazar
si no
   aceptar
```

Consulta por momento específico

Algoritmo:

1. Leer instante.
2. Buscar dos líneas consecutivas.
3. Determinar intervalo.

Pseudocódigo:

```
leer momento
leer línea actual
leer línea siguiente

si momento está entre ambas
   mostrar actual
```

Consulta por canción

Algoritmo:

Buscar texto en archivos de cada día.

Pseudocódigo:

```
para cada día
   abrir archivo
   buscar coincidencia
   mostrar hora
```

---

6. Pruebas realizadas

Se realizaron pruebas funcionales.

Caso 1: Generación de semana

Entrada:

- canciones.txt
- shows.txt
- publicidad.txt

Resultado:

Se generaron 7 archivos correctamente.

Comportamiento:

Correcto.

Caso 2: Consulta por día

Entrada:

día = lunes

Resultado:

Se imprimió toda la programación.

Comportamiento:

Correcto.

Caso 3: Consulta por momento específico

Entrada:

día = martes
hora = 08 15 30

Resultado:

Se mostró correctamente el contenido activo.

Comportamiento:

Correcto.

Caso 4: Consulta por canción

Entrada:

nombre = canción específica

Resultado:

Se listaron todas sus apariciones.

Comportamiento:

Correcto.

Caso 5: Control de repetición

Se verificó que una canción no apareciera dentro de 4 horas.

Resultado:

Restricción cumplida.

---

7. Conclusiones

Durante el desarrollo del proyecto se aplicaron conceptos fundamentales de programación en lenguaje C:

- archivos
- arreglos
- registros
- modularización
- funciones

La modularización permitió una mejora significativa respecto a la primera versión monolítica del código.

Las principales dificultades encontradas fueron:

- manejo de tiempos en segundos
- validación de repeticiones
- ubicación correcta de shows

Estas dificultades fueron resueltas mediante:

- funciones auxiliares
- validaciones
- separación lógica por módulos

Como mejoras futuras se podrían implementar:

- persistencia de configuración
- optimización del algoritmo de selección
- estadísticas de reproducción